

Sessão 3.1: O Pacote dplyr para Manipulação de Dados

Norberto Jorge Gonçalves

6 de Julho de 2025

Contents

1. O Pacote dplyr: A Ferramenta Essencial para Manipulação de Dados	1
1.1. Instalar e Carregar o dplyr	1
1.2. O Operador Pipe (%>%)	2
1.3.1. select(): Selecionar Colunas	2
1.3.2. filter(): Filtrar Linhas	3
1.3.3. mutate(): Criar/Modificar Colunas	4
1.3.4. arrange(): Ordenar Linhas	4
1.3.5. group_by() e summarise(): Agrupar e Resumir Dados	5

1. O Pacote dplyr: A Ferramenta Essencial para Manipulação de Dados

O dplyr é um pacote do R que faz parte do “tidyverse”, uma coleção de pacotes desenhados para a ciência de dados. Ele oferece um conjunto de funções (chamadas “verbos”) que tornam a manipulação de data frames muito mais intuitiva e eficiente.

1.1. Instalar e Carregar o dplyr

Primeiro, precisamos de instalar o pacote (apenas uma vez no vosso computador) e depois carregá-lo em cada nova sessão R que o vamos usar.

```
# Instalem o pacote dplyr (descomentem a linha e executem-na)
# install.packages("dplyr")
```

```
# Carreguem o pacote dplyr
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

1.2. O Operador Pipe (%>%)

O dplyr é frequentemente usado com o operador “pipe” (%>%). Este operador permite “encadear” operações, tornando o código mais legível. Pensem nele como “e depois”. Em vez de: `resultado <- funcao2(funcao1(dados))` Com o pipe: `resultado <- dados %>% funcao1() %>% funcao2()` Para criar o pipe no RStudio, usem Ctrl + Shift + M (Windows/Linux) ou Cmd + Shift + M (Mac). ## 1.3. Verbos Principais do dplyr Vamos usar o dataset iris para demonstrar os verbos.

```
data(iris)
head(iris)
```

##	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
## 1	5.1	3.5	1.4	0.2	setosa
## 2	4.9	3.0	1.4	0.2	setosa
## 3	4.7	3.2	1.3	0.2	setosa
## 4	4.6	3.1	1.5	0.2	setosa
## 5	5.0	3.6	1.4	0.2	setosa
## 6	5.4	3.9	1.7	0.4	setosa

1.3.1. select(): Selecionar Colunas

A função select() permite escolher colunas pelo nome.

```
# Selecionar apenas as colunas Sepal.Length e Species
iris_selecionado <- iris %>%
  select(Sepal.Length, Species)
head(iris_selecionado)
```

##	Sepal.Length	Species
## 1	5.1	setosa
## 2	4.9	setosa
## 3	4.7	setosa
## 4	4.6	setosa
## 5	5.0	setosa
## 6	5.4	setosa

```
# Selecionar todas as colunas EXCETO Petal.Width
iris_sem_petal_width <- iris %>%
  select(-Petal.Width)
head(iris_sem_petal_width)
```

##	Sepal.Length	Sepal.Width	Petal.Length	Species
## 1	5.1	3.5	1.4	setosa
## 2	4.9	3.0	1.4	setosa
## 3	4.7	3.2	1.3	setosa
## 4	4.6	3.1	1.5	setosa
## 5	5.0	3.6	1.4	setosa
## 6	5.4	3.9	1.7	setosa

```
# Selecionar colunas que começam com "Sepal"
iris_sepal <- iris %>%
  select(starts_with("Sepal"))
head(iris_sepal)
```

##	Sepal.Length	Sepal.Width
## 1	5.1	3.5
## 2	4.9	3.0
## 3	4.7	3.2

```
## 4      4.6      3.1
## 5      5.0      3.6
## 6      5.4      3.9
```

1.3.2. filter(): Filtrar Linhas

A função `filter()` permite selecionar linhas com base em condições lógicas. É semelhante à seleção de linhas com `[]` que vimos na Sessão 2, mas mais legível.

```
# Filtrar flores da espécie "virginica"
iris_virginica <- iris %>%
  filter(Species == "virginica")
head(iris_virginica)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width  Species
## 1         6.3         3.3         6.0         2.5 virginica
## 2         5.8         2.7         5.1         1.9 virginica
## 3         7.1         3.0         5.9         2.1 virginica
## 4         6.3         2.9         5.6         1.8 virginica
## 5         6.5         3.0         5.8         2.2 virginica
## 6         7.6         3.0         6.6         2.1 virginica
```

```
nrow(iris_virginica)
```

```
## [1] 50
```

```
# Filtrar flores com Sepal.Length maior que 7
iris_sepal_longo <- iris %>%
  filter(Sepal.Length > 7.0)
head(iris_sepal_longo)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width  Species
## 1         7.1         3.0         5.9         2.1 virginica
## 2         7.6         3.0         6.6         2.1 virginica
## 3         7.3         2.9         6.3         1.8 virginica
## 4         7.2         3.6         6.1         2.5 virginica
## 5         7.7         3.8         6.7         2.2 virginica
## 6         7.7         2.6         6.9         2.3 virginica
```

```
# Filtrar flores da espécie "setosa" E com Petal.Width menor que 0.3
iris_setosa_pequena <- iris %>%
  filter(Species == "setosa" & Petal.Width < 0.3)
head(iris_setosa_pequena)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5         1.4         0.2  setosa
## 2         4.9         3.0         1.4         0.2  setosa
## 3         4.7         3.2         1.3         0.2  setosa
## 4         4.6         3.1         1.5         0.2  setosa
## 5         5.0         3.6         1.4         0.2  setosa
## 6         5.0         3.4         1.5         0.2  setosa
```

```
# Filtrar flores da espécie "setosa" OU "versicolor"
iris_setosa_versicolor <- iris %>%
  filter(Species %in% c("setosa", "versicolor"))
head(iris_setosa_versicolor)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
```

```
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

```
nrow(iris_setosa_versicolor)
```

```
## [1] 100
```

1.3.3. mutate(): Criar/Modificar Colunas

A função mutate() permite criar novas colunas ou modificar colunas existentes.

```
# Criar uma nova coluna 'Sepal.Area' (Length * Width)
iris_com_area <- iris %>%
  mutate(Sepal.Area = Sepal.Length * Sepal.Width)
head(iris_com_area)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species Sepal.Area
## 1      5.1      3.5      1.4      0.2 setosa      17.85
## 2      4.9      3.0      1.4      0.2 setosa      14.70
## 3      4.7      3.2      1.3      0.2 setosa      15.04
## 4      4.6      3.1      1.5      0.2 setosa      14.26
## 5      5.0      3.6      1.4      0.2 setosa      18.00
## 6      5.4      3.9      1.7      0.4 setosa      21.06
```

```
# Criar uma coluna categórica 'Sepal.Length.Category'
iris_com_categoria <- iris %>%
  mutate(Sepal.Length.Category = ifelse(Sepal.Length > 6, "Longo", "Curto"))
head(iris_com_categoria)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
##   Sepal.Length.Category
## 1             Curto
## 2             Curto
## 3             Curto
## 4             Curto
## 5             Curto
## 6             Curto
```

1.3.4. arrange(): Ordenar Linhas

A função arrange() permite ordenar as linhas do data frame com base nos valores de uma ou mais colunas.

```
# Ordenar por Sepal.Length em ordem crescente (padrão)
iris_ordenado_asc <- iris %>%
  arrange(Sepal.Length)
head(iris_ordenado_asc)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
```

```
## 1      4.3      3.0      1.1      0.1 setosa
## 2      4.4      2.9      1.4      0.2 setosa
## 3      4.4      3.0      1.3      0.2 setosa
## 4      4.4      3.2      1.3      0.2 setosa
## 5      4.5      2.3      1.3      0.3 setosa
## 6      4.6      3.1      1.5      0.2 setosa
```

```
# Ordenar por Sepal.Length em ordem decrescente
iris_ordenado_desc <- iris %>%
  arrange(desc(Sepal.Length))
head(iris_ordenado_desc)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      7.9      3.8      6.4      2.0 virginica
## 2      7.7      3.8      6.7      2.2 virginica
## 3      7.7      2.6      6.9      2.3 virginica
## 4      7.7      2.8      6.7      2.0 virginica
## 5      7.7      3.0      6.1      2.3 virginica
## 6      7.6      3.0      6.6      2.1 virginica
```

```
# Ordenar por Species (crescente) e depois por Sepal.Length (decrescente)
iris_ordenado_multi <- iris %>%
  arrange(Species, desc(Sepal.Length))
head(iris_ordenado_multi)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.8      4.0      1.2      0.2 setosa
## 2      5.7      4.4      1.5      0.4 setosa
## 3      5.7      3.8      1.7      0.3 setosa
## 4      5.5      4.2      1.4      0.2 setosa
## 5      5.5      3.5      1.3      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

1.3.5. group_by() e summarise(): Agrupar e Resumir Dados

Estas duas funções são usadas em conjunto para realizar análises por grupo. * group_by(): Agrupa o data frame por uma ou mais colunas. * summarise(): Calcula estatísticas de resumo para cada grupo.

```
# Calcular a média do Sepal.Length por espécie
media_sepal_por_especie <- iris %>%
  group_by(Species) %>%
  summarise(Media_Sepal.Length = mean(Sepal.Length))
print(media_sepal_por_especie)
```

```
## # A tibble: 3 x 2
##   Species      Media_Sepal.Length
##   <fct>          <dbl>
## 1 setosa          5.01
## 2 versicolor     5.94
## 3 virginica       6.59
```

```
# Calcular a média e o desvio padrão do Petal.Width por espécie
resumo_petala_por_especie <- iris %>%
  group_by(Species) %>%
  summarise(
    Media_Petal.Width = mean(Petal.Width),
    DesvioPadrao_Petal.Width = sd(Petal.Width),
```

```

    Numero_Observacoes = n() # n() conta o número de observações no grupo
  )
print(resumo_petala_por_especie)

```

```

## # A tibble: 3 x 4
##   Species      Media_Petal.Width DesvioPadrao_Petal.Width Numero_Observacoes
##   <fct>          <dbl>          <dbl>          <int>
## 1 setosa          0.246          0.105           50
## 2 versicolor      1.33          0.198           50
## 3 virginica       2.03          0.275           50

```

Parabéns! Acabaram de aprender os verbos mais importantes do dplyr!